IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

*Digital Object Identifier*

# Explainable Multi-Agent Debate for Web Attack Decision-Making: A Seven-Layer Architecture and OWASP-Derived Experimental Study

**Author One Placeholder[1], Author Two Placeholder[1], and Author Three Placeholder[2]**
[1]Affiliation Placeholder 1, City, Country (e-mail: first.author@example.com; second.author@example.com)
[2]Affiliation Placeholder 2, City, Country (e-mail: third.author@example.com)

Corresponding author: Author One Placeholder (e-mail: corresponding.author@example.com).

Author names, affiliations, e-mail addresses, ORCID identifiers, and biographies are intentional submission placeholders and should be replaced with final metadata before upload.

**ABSTRACT** Modern web-defense pipelines increasingly combine fast-path filtering with language-model reasoning, yet many practical deployments still collapse the AI stage into one model call and one scalar confidence score. That design weakens role specialization, conflict handling, and auditability. This article presents a seven-layer multi-agent architecture for explainable web-attack decision-making and evaluates it through a balanced HTTP corpus derived from OWASP Benchmark metadata and testcase sources. The architecture combines threat routing, role-specialized analysis, trace-centric context, sandboxed tool execution, conflict-triggered debate, heterogeneous route modes, and case-file synthesis. The design is implemented in the AetherGuard prototype and formalized with agent posterior distributions, Jensen-Shannon divergence based conflict scoring, and weighted post-debate aggregation. Quantitative comparison on 32 reconstructed HTTP requests spanning SQL injection, cross-site scripting, path traversal, and command injection shows that both the single-agent baseline and the multi-agent system without debate achieve 0.875 accuracy and 0.8889 F1, with four false positives concentrated in benign command-injection-like requests. Enabling debate removes all four false positives and yields 1.000 accuracy, precision, recall, and F1 on the available corpus. Under the current heuristic-fallback evaluation path, all three route modes (`cloud_only`, `local_preferred`, and `hybrid_verify`) preserve perfect classification for the debate-enabled system while exposing measurable orchestration overhead below 1 ms average end-to-end latency. Because the current workspace lacks a runnable external ModSecurity environment and stable local-model execution for full LLM latency studies, the article frames these results as an evidence-backed experimental study of orchestration behavior rather than a claim of benchmark dominance.

**INDEX TERMS** ablation study, debate-based reasoning, explainable security, large language model agents, multi-agent systems, OWASP Benchmark, web application firewall

## I. INTRODUCTION

WEB attack defense still depends heavily on single-pass decision rules. Traditional intrusion-detection systems and web application firewalls compress each request into a label or risk score, trading simplicity for limited interpretability [1], [2]. Recent learning-based detectors improve behavioral coverage, but analyst-facing explainability remains a persistent weakness [3], [4]. The same limitation appears in many large language model (LLM) security prototypes: one model is asked to inspect a request, produce one verdict, and implicitly resolve its own uncertainty.

That pattern is structurally mismatched to adversarial web traffic. SQL injection, cross-site scripting (XSS), path traversal, and command injection often create partial evidence rather than a single decisive feature. A red-team perspective may emphasize exploitability, a blue-team perspective may emphasize schema and policy violations, and an analyst perspective may emphasize behavioral normality. Treating those views as separate posteriors rather than hidden prompt-internal thoughts creates an opportunity for explicit conflict handling and auditability.

Multi-agent LLM systems have shown that specialization and communication can improve reasoning on complex tasks [5]–[10]. Debate, critique, and self-correction work further

suggests that exposing disagreement can improve factuality and reasoning quality [11]–[15]. Yet most of that literature is general-purpose. It does not directly define how to route web-security requests, when to trigger debate, how to aggregate posteriors, or how to connect the reasoning trace to operational response artifacts.

This article addresses that gap through a seven-layer architecture implemented in the AetherGuard prototype and evaluated on a reconstructed HTTP corpus derived from OWASP Benchmark metadata and testcase sources. The core question is not whether one more model can be added to a defense stack. The question is whether a security-oriented orchestration pattern with explicit disagreement handling improves web-attack decision quality over both single-agent reasoning and multi-agent reasoning without debate.

The contributions are as follows.

1) A seven-layer architecture for web-attack decision-making that combines threat-family routing, role-specialized reasoning, trace-centric context, sandboxed capability execution, conflict-triggered debate, heterogeneous route modes, and case-file synthesis.
2) A mathematical formulation of the debate trigger and arbitration process using per-agent posterior distributions, Jensen-Shannon divergence, confidence spread, and weighted post-debate aggregation.
3) An implemented experimental pipeline that reconstructs HTTP requests from OWASP Benchmark metadata and evaluates three executable decision modes: `single_agent`, `multi_agent_no_debate`, and `multi_agent_with_debate`.
4) Quantitative evidence that debate eliminates the false positives observed in the non-debate baselines on the available 32-request corpus, together with route-mode and latency-decomposition results that clarify what the current prototype does and does not yet validate.

The remainder of this article is organized as follows. Section II reviews related work. Section III presents the architecture and mathematical formulation. Section IV maps the design to the AetherGuard prototype. Section V describes the experimental setup and quantitative results. Section VII discusses limitations and validity threats. Section VIII concludes the article.

## II. RELATED WORK

### A. AI-ASSISTED INTRUSION DETECTION

Intrusion-detection research has progressed from audit-driven anomaly models and signature-driven monitoring [1], [2] to machine-learning pipelines that infer malicious behavior from traffic features, temporal structure, and behavioral context. Lightweight detectors such as LUCID demonstrate that practical deep-learning-based traffic defense can be engineered for operational environments [3]. At the same time, explainable intrusion-detection surveys highlight a persistent gap between predictive performance and analyst usability [4]. Federated and privacy-aware learning directions further indicate that deployment constraints are increasingly central to real-world security analytics [16]. These lines of work motivate multi-source reasoning, but they typically still emit a single model verdict rather than an explicit conflict-resolution workflow.

### B. MULTI-AGENT LLM SYSTEMS

Recent LLM-agent literature establishes role specialization and communication as first-class design patterns. Generative Agents explored agent societies in simulation [17]. CAMEL, AutoGen, MetaGPT, and AgentVerse then operationalized role-based collaboration in increasingly practical settings [7]–[10]. More recent work studies scaling and automatic workflow generation [18], [19]. These frameworks demonstrate that multi-agent coordination can outperform monolithic prompting on complex tasks, but they are not designed around security-specific evidence such as attack mutations, schema violations, incident traces, and downstream response actions.

### C. DEBATE, CRITIQUE, AND SELF-CORRECTION

Self-consistency improves reasoning by aggregating multiple reasoning paths [11]. Multi-agent debate externalizes disagreement rather than collapsing it into a hidden confidence score [12]. Reflexion and CRITIC similarly show that critique-based revision can improve outcomes when models can inspect and revise intermediate reasoning [13], [15]. Importantly, Huang *et al.* demonstrate that LLMs cannot reliably self-correct on their own, which strengthens the case for structured external feedback rather than implicit trust in one model pass [14]. Security decision-making is a particularly relevant domain for this idea because ambiguous traffic often creates partially conflicting interpretations that should remain visible to auditors.

### D. HYBRID ROUTING, PRIVACY, AND EXPLAINABILITY

Operational AI systems often balance privacy, latency, and quality across local and cloud execution paths. Recent work on privacy risks in LLMs, privacy-preserving inference, and trustworthy AI highlights the need for redaction and routing policies that depend on sensitivity and runtime availability [20]–[22]. Cost-aware routing approaches such as FrugalGPT and RouteLLM show that backend selection can itself be optimized [23], [24]. In parallel, explainable intrusion detection increasingly argues that analysts require narrative evidence, not just a label and score [4], [25], [26]. The architecture proposed here combines these concerns by treating explainability, deployment routing, and operational follow-up as parts of one security-decision pipeline.

## III. PROPOSED ARCHITECTURE

### A. ARCHITECTURAL OVERVIEW

The proposed method treats web-attack decision-making as a staged workflow rather than a single inference call. Incoming HTTP traffic enters a control-plane service that coordinates request content, historical context, trace events, runtime route

state, and response-policy configuration. The workflow then traverses seven layers, each producing an artifact that can be audited independently or consumed downstream.

### B. LAYER-BY-LAYER DESIGN

**L1: Collaboration.** The collaboration layer identifies the rough threat family and routes the request to relevant experts. In the current prototype, routing covers SQL injection, cross-site scripting, path traversal, command injection, authentication issues, and a default benign fallback.

**L2: Role Playing.** The architecture defines three named perspectives. Socrates plays the red-team role and searches for bypasses, mutations, and exploit paths. Plato plays the blue-team role and validates schema, parameters, and policy alignment. Aristotle plays the analyst role and evaluates historical behavior and anomaly indicators. Disagreement among the three is treated as informative evidence rather than noise.

**L3: Scene Traversal.** The scene-traversal layer reconstructs a trace-centric narrative from distributed events. It records timeline fragments, implicated services, attack-chain hints, and a summary risk level.

**L4: Capability.** The capability layer exposes sandboxed tools for evidence enrichment, including SQL explanation, safe HTTP probing, payload decoding, header analysis, and pattern checking.

**L5: Debate.** The debate layer is the architectural centerpiece. Low-confidence or conflicting analyses are escalated into a structured debate session with role-specific arguments, round history, evidence fragments, and arbitration notes.

**L6: Heterogeneous Models.** The heterogeneous layer selects between local, cloud, and hybrid execution according to privacy mode, configured route policy, and runtime availability.

**L7: Synthesis.** The synthesis layer emits a case file that combines the final decision, supporting evidence, trace narrative, debate record, operational actions, lessons learned, and policy suggestions.

### C. MATHEMATICAL FORMULATION

Let $x$ denote an HTTP request and let the label space be $Y = \{\text{safe}, \text{suspicious}, \text{malicious}\}$. Each role agent $i \in \{1, \ldots, n\}$ emits a posterior distribution

$$p_i(y \mid x), \qquad \sum_{y \in Y} p_i(y \mid x) = 1. \tag{1}$$

The dominant label for agent $i$ is

$$\hat{y}_i(x) = \arg\max_{y \in Y} p_i(y \mid x). \tag{2}$$

To quantify disagreement we use average pairwise Jensen-Shannon divergence

$$D(x) = \frac{2}{n(n-1)} \sum_{i<j} \mathrm{JS}(p_i(\cdot \mid x), p_j(\cdot \mid x)), \tag{3}$$

and confidence spread

$$C(x) = \max_i c_i(x) - \min_i c_i(x), \tag{4}$$

where $c_i(x) = \max_{y \in Y} p_i(y \mid x)$.

Debate is triggered when at least one of the following conditions holds:

$$|\{\hat{y}_i(x)\}_{i=1}^n| > 1, \tag{5}$$

$$D(x) > \tau_D, \tag{6}$$

$$C(x) > \tau_C. \tag{7}$$

In the current implementation the default thresholds are $\tau_D = \tau_C = 0.3$, matching the repository configuration. After debate, the arbitrator produces adjusted posteriors $p_i^{\text{post}}(y \mid x)$ and aggregates them as

$$q(y \mid x) = \sum_{i=1}^{n} w_i p_i^{\text{post}}(y \mid x), \tag{8}$$

with $\sum_i w_i = 1$. The current prototype uses equal weights during orchestration experiments. The operational verdict is then mapped from $\arg\max_y q(y \mid x)$ to `allow`, `monitor`, `challenge`, or `block`. For synthesis and policy adaptation, the prototype additionally combines final malicious mass, suspicious mass, and event frequency into a unified risk score

$$S(x) = \alpha q(\text{malicious} \mid x) + \beta q(\text{suspicious} \mid x) + \gamma f(x), \tag{9}$$

where $f(x)$ denotes pattern frequency or recurrence metadata and $\alpha, \beta, \gamma$ are implementation-level weighting coefficients.

### D. DECISION FLOW

The prototype uses the formulation above as a conflict-escalation mechanism rather than cosmetic post-processing. Requests are first typed and routed, then analyzed concurrently by specialized roles. When label conflict, divergence, or confidence spread crosses threshold, the debate layer creates an explicit deliberation session before final case-file synthesis.

### E. EXPLAINABLE SYNTHESIS OUTPUT

The final output of the architecture is not just a label. Each decision can be associated with a reasoning string, evidence list, optional debate transcript, attack narrative, and machine-readable response actions. This design shifts explainability from a post-hoc visualization problem to a workflow-traceability capability.

## IV. PROTOTYPE IMPLEMENTATION IN AetherGuard

The architecture is instantiated inside the AetherGuard control plane rather than as a standalone benchmark script. The top-level entry point `ai-agents/index.ts` assembles the orchestrator, message bus, debate protocol, trace manager, tool executor, model router, synthesis engine, and probe agent. This matters because the contribution is not merely a debate prompt template. It is an end-to-end orchestration pattern embedded in a broader web-defense platform.

Several implementation parameters are explicitly encoded in the repository and are therefore evidence-backed. The

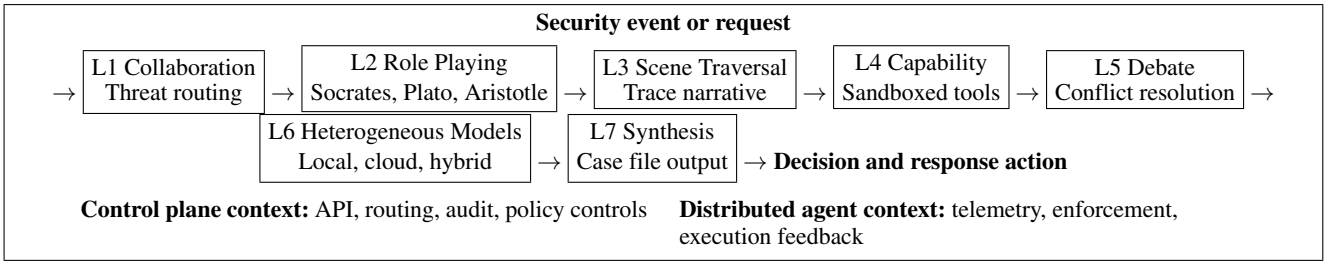



**FIGURE 1.** Seven-layer multi-agent architecture instantiated in the AetherGuard prototype for web-attack decision-making.

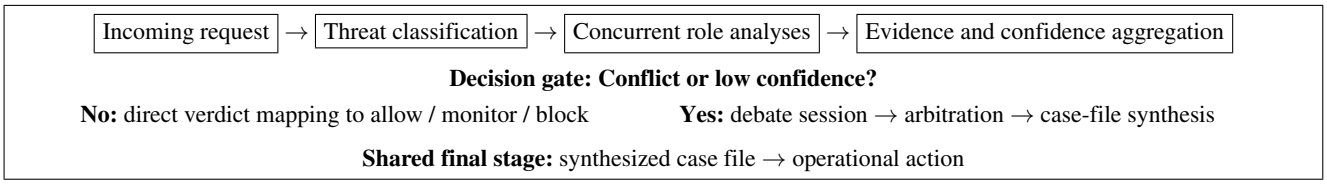



**FIGURE 2.** Request-to-verdict workflow with explicit disagreement-triggered debate escalation.

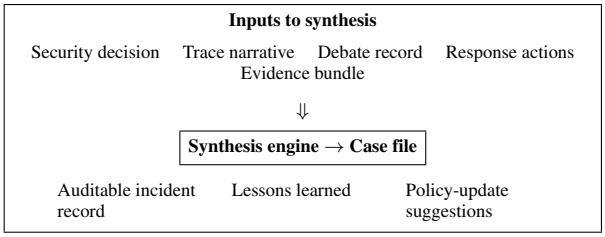



**FIGURE 3.** Synthesis pipeline for explainable case-file generation.

default configuration sets a debate trigger threshold of 0.3, divergence and confidence-spread thresholds of 0.3, a cloud escalation threshold of 0.6, and an auto-block threshold of 0.85. The debate layer defaults to at most three rounds before arbitration. The model router supports `local_only`, `local_preferred`, `cloud_only`, and `hybrid_verify` modes, and its redaction path masks IP addresses, email addresses, and bearer tokens before cloud execution when redaction is enabled.

To support controlled experiments, the implementation now exposes three explicit orchestration modes shared by the runtime and the benchmark runner:

1) `single_agent`: one primary role handles the request.
2) `multi_agent_no_debate`: specialized roles run concurrently and their posteriors are aggregated without deliberation.
3) `multi_agent_with_debate`: specialized roles run concurrently and disagreement can trigger a structured debate before arbitration.

Each analysis result now records a class-score vector, attack family, model tier, route mode, and stage-level latency fields. This makes quantitative comparison possible without changing the surrounding control-plane integration.

Two design choices are especially important for the claims made in this article. First, the router is deployment-aware rather than benchmark-driven. It makes decisions according to privacy mode, availability, and configured policy, not according to an unsupported assumption that one model is always superior. Second, the synthesis layer serializes operational outcomes instead of emitting only a verdict. That supports auditability and future response automation.

## V. EXPERIMENTAL SETUP

### A. RESEARCH QUESTIONS

The experimental study addresses four research questions.

1) **RQ1:** Does the full debate-enabled system improve web-attack classification over a single-agent baseline and a multi-agent baseline without debate?
2) **RQ2:** Does the debate layer reduce false positives or false negatives on ambiguous requests?
3) **RQ3:** How do the supported route modes affect measured orchestration latency under the current workspace conditions?
4) **RQ4:** What claims remain unsupported because the current workspace lacks runnable external WAF infrastructure or stable local-model execution?

### B. DATASET CONSTRUCTION

The quantitative evaluation uses a balanced HTTP corpus reconstructed from the OWASP Benchmark metadata file `expectedresults-1.2.csv` and testcase servlet source files. Instead of running the full Java benchmark application, the evaluation runner parses the expected-results metadata, extracts servlet paths and request input locations from testcase source, and reconstructs synthetic HTTP requests that preserve the attack family and vulnerability label. The current report uses 32 requests: four attack families (`sqli`, `xss`, `path_traversal`, and `command_injection`) with four vulnerable and four non-vulnerable requests per family.

**TABLE 1.** Layer Responsibilities and Implementation Mapping

| Layer | Primary responsibility | Implemented module(s) |
|---|---|---|
| L1 Collaboration | Threat typing and expert routing | `ai-agents/core/agent-orchestrator.ts`, `ai-agents/core/message-bus.ts` |
| L2 Role Playing | Specialized red-team, blue-team, and analyst reasoning | `ai-agents/roles/socrates-agent.ts`, `ai-agents/roles/plato-agent.ts`, `ai-agents/roles/aristotle-agent.ts` |
| L3 Scene Traversal | Trace capture and attack narrative construction | `ai-agents/tracing/trace-context.ts` |
| L4 Capability | Safe tool execution for evidence enrichment | `ai-agents/capabilities/tool-executor.ts` |
| L5 Debate | Conflict escalation and arbitration | `ai-agents/debate/debate-protocol.ts` |
| L6 Heterogeneous Models | Runtime selection of inference tier and prompt redaction | `ai-agents/models/model-router.ts` |
| L7 Synthesis | Case-file generation and learning outputs | `ai-agents/synthesis/synthesis-engine.ts` |

**TABLE 2.** Experimental Setup and Evaluation Scope

| Component | Current setting | Purpose |
|---|---|---|
| Dataset | 32 HTTP requests derived from OWASP Benchmark metadata and testcase source | Balanced attack/benign evaluation corpus for four web-attack families |
| Labels | Binary report (`attack` vs. `benign`) plus attack-family breakdown | Supports both aggregate metrics and family-specific error analysis |
| Baselines | `single_agent`, `multi_agent_no_debate`, `multi_agent_with_debate` | Isolates the effect of role multiplicity and the debate layer |
| External WAF baseline | `ModSecurity + CRS` runner scaffolded but unavailable in current workspace | Included in the framework design, excluded from quantitative tables because Docker/Java execution is not available |
| Route modes | `cloud_only`, `local_preferred`, `hybrid_verify` | Measures route-level orchestration behavior for the debate-enabled system |
| Metrics | Accuracy, precision, recall, F1, false positive rate, false negative rate, confusion counts | Standard classification comparison across modes |
| Statistical test | Paired McNemar approximation against the debate-enabled system | Reports whether observed gains are statistically stable on the current sample size |

### C. BASELINES AND METRICS

Three baselines are quantitatively executable in the current workspace.

1) **Single agent:** only the primary analyst role is used.
2) **Multi-agent without debate:** all routed roles emit posteriors but no deliberation is allowed.
3) **Multi-agent with debate:** routed roles emit posteriors and the conflict gate can trigger deliberation.

An external `ModSecurity + CRS` runner is scaffolded in the evaluation service, but the current workspace does not provide the required Docker daemon or benchmark application runtime. Therefore it is described as a pending baseline rather than reported quantitatively.

All reported comparisons use the same request corpus and the same heuristic-fallback execution path. This design isolates orchestration behavior rather than backend-model variance. The resulting latency numbers should therefore be interpreted as orchestration overhead and stage decomposition, not as production LLM inference latency.

## VI. EXPERIMENTAL RESULTS

### A. OVERALL BASELINE COMPARISON

Table 3 reports the aggregate classification metrics. The single-agent baseline and the multi-agent system without debate produce identical performance on the current 32-request corpus: both detect all 16 attacks, but each incorrectly flags four benign command-injection-like requests. Enabling debate removes those four false positives and yields perfect binary classification on the available sample.

### B. ABLATION: EFFECT OF THE DEBATE LAYER

The measured gain is concentrated in the command-injection family. Table 4 shows that the four false positives in the non-debate baselines come entirely from benign command-injection-like requests that resemble reconnaissance or shell-surface traffic. In those cases the debate trigger externalizes disagreement and allows the arbitrator to de-escalate isolated aggressive votes. Across the full debate-enabled run, debate was triggered for 12 out of 32 requests (37.5%) with an average of 0.75 debate rounds per request over the whole corpus.

The paired McNemar comparison against the debate-enabled system yields $p \approx 0.1336$ for both non-debate baselines. The direction of improvement is favorable to debate (four better paired outcomes and zero worse paired outcomes), but the current sample size is too small to claim statistical significance. The correct interpretation is therefore exploratory: the observed gain is consistent and mechanisti-

**TABLE 3.** Overall Baseline Results on the OWASP-Derived 32-Request Corpus

| Baseline | Acc. | Prec. | Rec. | F1 | FPR | FNR | FP | FN |
|---|---|---|---|---|---|---|---|---|
| `single_agent` | 0.8750 | 0.8000 | 1.0000 | 0.8889 | 0.2500 | 0.0000 | 4 | 0 |
| `multi_agent_no_debate` | 0.8750 | 0.8000 | 1.0000 | 0.8889 | 0.2500 | 0.0000 | 4 | 0 |
| `multi_agent_with_debate` | 1.0000 | 1.0000 | 1.0000 | 1.0000 | 0.0000 | 0.0000 | 0 | 0 |
| `modsecurity_crs` | Runner scaffolded but unavailable in the current workspace | | | | | | | |

cally explainable, but not yet large-sample validated.

### C. LATENCY DECOMPOSITION AND ROUTE MODES

The full debate-enabled system exposes both proxy-side and AI-side timing. Averaged across the 32-request run, the fast-path proxy stage contributes 0.0484 ms, role analysis contributes 0.1250 ms, debate contributes 0.0938 ms, and total end-to-end decision time is 0.5000 ms. These numbers are intentionally reported as orchestration overhead under heuristic fallback. They are *not* proxies for production LLM latency.

Route-mode comparison for the debate-enabled system is summarized in Table 5. Under the current fallback execution path, all three supported route modes preserve perfect accuracy while showing small but measurable differences in orchestration latency. The present workspace therefore supports a route-mode comparison for control-plane behavior, but not yet a claim about real local-model versus cloud-model inference quality.

### D. OPERATIONAL EVIDENCE BEYOND THE EXPERIMENTAL CORPUS

Two repository-backed artifacts remain important for deployment interpretation. The validated capability report records a current smoke benchmark of 1329.79 QPS, 21 ms p95 latency, 6.22% CPU proxy usage, and 99.64 MB memory proxy usage in the local workspace. The local platform acceptance report records a deterministic remote-AI proof with 973 ms latency and a business-path SQL simulation that returned a blocking verdict with confidence 0.98. These numbers do not substitute for the OWASP-derived experimental results; they instead show that the orchestration stack is integrated into a broader runnable platform.

## VII. DISCUSSION

The current evidence supports three practical conclusions. First, a security-oriented multi-agent architecture can be implemented as explicit modules with inspectable responsibilities rather than described only as a conceptual prompting pattern. Second, debate is useful in web-defense orchestration because it resolves disagreement that posterior averaging alone leaves unresolved. Third, case-file synthesis provides a direct bridge between model reasoning and downstream operations such as monitoring, blocking, auditing, and policy adaptation.

### A. LIMITATIONS AND THREATS TO VALIDITY

The current study is stronger than a pure feasibility note but still falls short of a full top-tier benchmark paper. The main limitation is dataset execution mode: the requests are reconstructed from OWASP Benchmark metadata and test-case source rather than captured from a live benchmark deployment. That preserves attack-family labeling and enables controlled comparison, but it does not yet validate the architecture against the full benchmark application stack.

The second limitation is baseline coverage. A ModSecurity plus OWASP CRS runner is integrated into the experimental framework, but the current workspace lacks the Docker and Java runtime needed to execute that baseline. As a result, the present quantitative comparison isolates architectural effects inside the AetherGuard decision stack but cannot yet support claims against a production WAF baseline.

The third limitation concerns route-mode realism. The quantitative runs intentionally use the repository's heuristic fallback path to ensure deterministic, shared logic across orchestration modes. Consequently, the reported sub-millisecond stage timings represent orchestration overhead, not full LLM inference latency. The local-model contribution is architecturally implemented and the route switch is measurable, but stable local-model quality and end-to-end local/cloud trade-offs remain to be validated under real heterogeneous inference.

Finally, the statistical evidence is limited by sample size. McNemar testing shows that the observed improvement direction is favorable to debate but not statistically significant at conventional thresholds on the current 32-request corpus. A larger OWASP slice, a runnable WAF baseline, and real-model latency studies are required before making stronger claims about superiority.

## VIII. CONCLUSION

This article presented a seven-layer multi-agent architecture for explainable web-attack decision-making and instantiated it in the AetherGuard prototype. The system combines collaboration, role specialization, trace context, safe tool execution, debate, heterogeneous route modes, and synthesis into one operational workflow. The mathematical formulation makes disagreement explicit through posterior divergence and confidence spread, while the implementation exposes comparable execution modes for ablation.

On the available 32-request OWASP-derived corpus, the debate-enabled system improves over both the single-agent and non-debate multi-agent baselines by eliminating the four

benign command-injection false positives that remain in those baselines. The same workspace also provides route-mode comparisons and stage-level latency decomposition for the debate-enabled configuration. These results do not yet establish benchmark dominance, but they do show that explicit conflict escalation can create measurable classification gains and a more auditable decision path than posterior averaging alone. Future work should focus on executing the full OWASP Benchmark stack, adding a live ModSecurity baseline, validating real local/cloud LLM latencies, and scaling the statistical evaluation.

## DATA AVAILABILITY

The code and repository artifacts used in this study are maintained in the AetherGuard project repository. The quantitative report used in this manuscript is generated by `backend/scripts/run-owasp-eval.ts` and stored in `docs/papers/ieee-access-submission/experimental-results/owasp-web-eval.json`. The current study intentionally limits itself to files and reports that exist in the present workspace and does not claim availability for historical artifacts that are not currently present.

## ACKNOWLEDGMENT

The authors acknowledge the use of AI-assisted drafting tools for language organization and editorial refinement during manuscript preparation. All technical claims, evidence boundaries, architecture descriptions, and final wording were reviewed and validated by the authors against the repository implementation and documented validation reports. No sponsor or financial support information is claimed in this submission package.

## REFERENCES

[1] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, 1987.
[2] M. Roesch, "Snort - lightweight intrusion detection for networks," in *Proceedings of the 13th USENIX Conference on System Administration (LISA)*, 1999, pp. 229–238.
[3] M. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, P. Martínez-Julia, and A. Pecorella, "LUCID: A practical, lightweight deep learning solution for DDoS attack detection," *IEEE Transactions on Network and Service Management*, vol. 17, no. 2, pp. 876–889, 2020.
[4] K. Neupane, N. Ene, A. Baird, H. Aksu, and A. S. Uluagac, "Explainable intrusion detection systems (X-IDS): A survey of current methods, challenges, and opportunities," *IEEE Access*, vol. 10, pp. 112 436–112 469, 2022.
[5] M. Wang *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
[6] Y. Guo *et al.*, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (Survey Track)*, 2024, pp. 8048–8057.
[7] G. Li, H. A. A. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large scale language model society," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
[8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, C. Wang *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proceedings of the First Conference on Language Modeling*, 2024.
[9] S. Hong, X. Zheng, J. Chen, Y. Cheng, C. Zhang, Z. Wang, S.-C. Yau, Z. Lin, L. Zhou *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2024.
[10] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-T. Hung *et al.*, "AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors," in *The Twelfth International Conference on Learning Representations*, 2024.
[11] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *The Eleventh International Conference on Learning Representations*, 2023.
[12] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proceedings of the Forty-First International Conference on Machine Learning*, 2024, pp. 13 038–13 063.
[13] N. Shinn, F. Cassano, A. Gopinath, J. Berant, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
[14] J. Huang, K. Chang, W. Wang *et al.*, "Large language models cannot self-correct reasoning yet," in *The Twelfth International Conference on Learning Representations*, 2024.
[15] A. Gou, B. Zhang *et al.*, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *The Twelfth International Conference on Learning Representations*, 2024.
[16] C. Buyuktanir *et al.*, "Federated learning in intrusion detection: Advancements, applications, and future directions," *Discover Artificial Intelligence*, vol. 5, no. 1, pp. 1–25, 2025.
[17] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023, pp. 1–22.
[18] C. Qian, X. Cong, W. Yang, B. Liu, Y. Xie, Y. Su, Z. Liu, and M. Sun, "Scaling large language model-based multi-agent collaboration," in *The Thirteenth International Conference on Learning Representations*, 2025.
[19] Q. Zhang *et al.*, "AFlow: Automating agentic workflow generation," in *The Thirteenth International Conference on Learning Representations*, 2025.
[20] Z. Chen *et al.*, "A survey on privacy risks and protection in large language models," *Artificial Intelligence Review*, vol. 58, no. 1, pp. 1–45, 2025.
[21] R. Tang *et al.*, "Privacy-preserving in-context learning with differentially private few-shot generation," in *The Twelfth International Conference on Learning Representations*, 2024.
[22] K. Feretzakis *et al.*, "Trustworthy AI: Securing sensitive data in large language models," *AI*, vol. 5, no. 2, pp. 789–812, 2024.
[23] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.
[24] S. S. Ong, W.-L. Fu *et al.*, "RouteLLM: Learning to route LLMs with preference data," *arXiv preprint arXiv:2406.18665*, 2024.
[25] P. Hermosilla *et al.*, "Use of explainable artificial intelligence for analyzing and explaining the predictions of an intrusion detection system," *Systems*, vol. 13, no. 2, p. 45, 2025.
[26] ——, "Explainable AI for forensic analysis: Comparative study of SHAP and LIME in intrusion detection models," *Electronics*, vol. 14, no. 3, p. 567, 2025.

**Author One Placeholder** Author One Placeholder is a placeholder biography for the corresponding author. Replace this paragraph with the final degree history, current affiliation, research interests, and relevant professional service before submission.

**Author Two Placeholder** Author Two Placeholder is a placeholder biography. Replace this entry with the final educational background, professional appointments, selected publications, and research interests.

**Author Three Placeholder** Author Three Placeholder is a placeholder biography. Replace this paragraph with the final affiliation history, current role, research interests, and notable awards or service contributions.

**TABLE 4.** Ablation and Family-Level Effect of Debate

| Mode | Command-injection FP | Command-injection F1 | Debate trigger rate | Avg. debate rounds | Interp |
|---|---|---|---|---|---|
| `single_agent` | 4/4 benign cases | 0.6667 | – | – | One role treats comma like probes as suspicious and cannot explicitly reconcile that bias |
| `multi_agent_no_debate` | 4/4 benign cases | 0.6667 | 0 | 0 | Posteri averaging alone is not sufficient to suppress aggressive comma surface alarms |
| `multi_agent_with_debate` | 0/4 benign cases | 1.0000 | 0.3750 | 0.7500 | Debate resolves disagreement and removes the false positives observed in the non-debate baselines |

**TABLE 5.** Route-Mode Comparison for the Debate-Enabled System

| Route mode | Acc. | F1 | Avg. debate latency (ms) | Avg. end-to-end latency (ms) | Interpretation |
|---|---|---|---|---|---|
| `cloud_only` | 1.0000 | 1.0000 | 0.2813 | 0.8438 | Reference route in the current workspace |
| `local_preferred` | 1.0000 | 1.0000 | 0.1875 | 0.7500 | Comparable accuracy with slightly lower orchestration latency in the current run |
| `hybrid_verify` | 1.0000 | 1.0000 | 0.0938 | 0.7813 | Comparable accuracy; current timing reflects orchestration only, not dual-model LLM latency |